

Lecture Notes on Disk Management

Example of FCFS (First-Come, First-Served) HDD Scheduling

Given:

- Initial position of the disk head: 50
- Queue of pending disk requests (in the order they arrive): 98, 183, 37, 122, 14, 124, 65, 67

FCFS Scheduling Execution:

The disk head processes the requests in the exact order they arrive.

1. Move from **50** → **98** (Total movement: **48**)
2. Move from **98** → **183** (Total movement: **85**)
3. Move from **183** → **37** (Total movement: **146**)
4. Move from **37** → **122** (Total movement: **85**)
5. Move from **122** → **14** (Total movement: **108**)
6. Move from **14** → **124** (Total movement: **110**)
7. Move from **124** → **65** (Total movement: **59**)
8. Move from **65** → **67** (Total movement: **2**)

Total Head Movement Calculation:

$48+85+146+85+108+110+59+2=643$ cylinders

Explanation:

- FCFS serves requests **in the exact order they arrive**.
- This approach is **fair** but can lead to **large seek times** if requests are scattered across the disk.
- In this example, the total head movement is **643 cylinders**.

Visualization of Head Movement:

50 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Example of SSTF (Shortest Seek Time First) HDD Scheduling

Given:

- Initial position of the disk head: 50
- Queue of pending disk requests (unordered): 98, 183, 37, 122, 14, 124, 65, 67
- Disk size range: 0 to 199

SSTF Scheduling Execution:

SSTF (Shortest Seek Time First) selects the request **closest to the current head position** to minimize seek time.

1. **Initial position: 50**
 - Closest request = 37 ($|50 - 37| = 13$)
 - **Move from 50 → 37 (Total movement: 13)**
2. **Current position: 37**
 - Closest request = 14 ($|37 - 14| = 23$)
 - **Move from 37 → 14 (Total movement: 23)**
3. **Current position: 14**
 - Closest request = 65 ($|14 - 65| = 51$)
 - **Move from 14 → 65 (Total movement: 51)**
4. **Current position: 65**
 - Closest request = 67 ($|65 - 67| = 2$)
 - **Move from 65 → 67 (Total movement: 2)**
5. **Current position: 67**
 - Closest request = 98 ($|67 - 98| = 31$)
 - **Move from 67 → 98 (Total movement: 31)**
6. **Current position: 98**
 - Closest request = 122 ($|98 - 122| = 24$)
 - **Move from 98 → 122 (Total movement: 24)**
7. **Current position: 122**
 - Closest request = 124 ($|122 - 124| = 2$)
 - **Move from 122 → 124 (Total movement: 2)**
8. **Current position: 124**
 - Closest request = 183 ($|124 - 183| = 59$)
 - **Move from 124 → 183 (Total movement: 59)**

Total Head Movement Calculation:

$13+23+51+2+31+24+2+59=205$ cylinders

Explanation:

- SSTF selects the nearest request at each step, reducing seek time significantly.
- **It is more efficient than FCFS, SCAN, and C-SCAN in minimizing head movement.**
- **However, it may lead to starvation** because distant requests might be delayed indefinitely if closer requests keep arriving.

Visualization of Head Movement:

50 → 37 → 14 → 65 → 67 → 98 → 122 → 124 → 183

Example of SCAN (Elevator Algorithm) HDD Scheduling

Given:

- Initial position of the disk head: 50
- Queue of pending disk requests (unordered): 98, 183, 37, 122, 14, 124, 65, 67
- Disk head movement direction: Moving towards higher track numbers
- Assume the disk has a range of tracks from 0 to 199

SCAN Scheduling Execution:

The disk head **moves towards one end (higher track numbers), servicing requests along the way, then reverses direction.**

1. Moving Right (Higher Numbers):

- 50 → 65 (Total movement: 15)
- 65 → 67 (Total movement: 2)
- 67 → 98 (Total movement: 31)
- 98 → 122 (Total movement: 24)
- 122 → 124 (Total movement: 2)
- 124 → 183 (Total movement: 59)
- 183 → 199 (Total movement: 16) (*Continue to the highest track before reversing*)

2. Reversing Left (Lower Numbers):

- 199 → 37 (Total movement: 162)
- 37 → 14 (Total movement: 23)

Total Head Movement Calculation:

$15+2+31+24+2+59+16+162+23=334$ cylinders

Explanation:

- SCAN moves in one direction, services all requests, then reverses (like an elevator).
- Less seek time than FCFS since it avoids back-and-forth movements.
- Total head movement is 334 cylinders (which is better than 643 in FCFS).

Visualization of Head Movement:

50 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14

Example of C-SCAN (Circular SCAN) HDD Scheduling

Given:

- Initial position of the disk head: 50
- Queue of pending disk requests (unordered): 98, 183, 37, 122, 14, 124, 65, 67
- Disk head movement direction: Moving towards higher track numbers
- Assume the disk has a range of tracks from 0 to 199

C-SCAN Scheduling Execution:

C-SCAN (Circular SCAN) moves in one direction (right), services requests, reaches the highest track, then jumps back to the lowest track without servicing any requests on the way back.

1. Moving Right (Higher Numbers):

- 50 → 65 (Total movement: 15)
- 65 → 67 (Total movement: 2)
- 67 → 98 (Total movement: 31)
- 98 → 122 (Total movement: 24)
- 122 → 124 (Total movement: 2)
- 124 → 183 (Total movement: 59)
- 183 → 199 (Total movement: 16) *(Continue to the highest track before jumping to 0)*

2. Jump to 0 (Without Servicing Requests)

- 199 → 0 (Total movement: 199) *(C-SCAN always jumps to the lowest track)*

3. Moving Right Again (Servicing Lower Requests):

- 0 → 14 (Total movement: 14)
- 14 → 37 (Total movement: 23)

Total Head Movement Calculation:

$15+2+31+24+2+59+16+199+14+23=385$ cylinders

Explanation:

- C-SCAN moves in one direction, reaches the highest track, jumps to the lowest track, and continues servicing.
- Prevents starvation of requests by giving all processes fair access.
- Better than SCAN in terms of uniform wait times.
- Total head movement is 385 cylinders (which is still better than 643 in FCFS but slightly worse than 334 in SCAN).

Visualization of Head Movement:

50 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → (Jump to 0) → 14 → 37

How to Implement Rotation Latency in Scheduling Algorithms?

Each request not only has a **track number** but also a **sector number** (0- sectors per track).

1. Step 1: Compute Seek Time

- Use traditional scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN, etc.) to determine **which track to access next**.

2. Step 2: Compute Rotational Latency

- Assume **the disk rotates at X RPM (Rotations Per Minute)**.
- **Rotational latency = time for one full rotation ÷ 2 (on average)**

$$T_{rot} = \frac{60}{2 \times RPM} \text{ seconds}$$

- If sector numbers are known, calculate the **exact rotation delay** based on the difference between the current sector and the requested sector.

3. Step 3: Compute Transfer Time

- Once the desired sector is under the head, the transfer time is calculated based on data rate and sector size.

Example Implementation of SSTF with Rotation Latency

Given:

- Disk with 200 tracks, 8 sectors per track.
- Disk rotates at 7200 RPM.
- Current position: Track 50, Sector 3
- Queue: (Track, Sector): (37, 6), (14, 2), (65, 4), (67, 7), (98, 5)
- Assume seek time per track = 0.1ms

Step 1: Select the Nearest Track (SSTF Algorithm)

- Nearest track: 37
- Seek time = $(50 - 37) \times 0.1\text{ms} = 1.3\text{ms}$

Step 2: Compute Rotational Latency

- Current sector = 3, requested sector = 6
- Disk rotation time per full revolution: $T_{full} = \frac{60}{7200} = 8.33 \text{ ms}$
- Rotational latency (time for 3 sectors to pass): $T_{rot} = \frac{3}{8} \times 8.33 = 3.125 \text{ ms}$
- Total Time to Access Track 37, Sector 6: $T_{total} = T_{seek} + T_{rot} = 1.3 + 3.125 = 4.425 \text{ ms}$

Effective Transfer Rate (ETR)

The Effective Transfer Rate (ETR) represents the actual data throughput of a Hard Disk Drive (HDD), considering all delays such as seek time, rotational latency, and transfer time.

When accessing data from a storage device, **two main components** affect the time taken:

1. **Access Latency (or Access Time):**

The time it takes to locate and start accessing the data (includes seek time, rotational delay, etc.).
→ Measured in milliseconds (ms) or seconds (s).

2. **Transfer Time:**

The time it takes actually to transfer the data once access has begun.

$$\text{Transfer Time} = \frac{\text{Amount of Data}}{\text{Transfer Rate of Device}}$$

Formula:

The **effective transfer rate** (i.e., how fast the data appears to be delivered, *including* overhead) is:

$$\text{Effective Transfer Rate} = \frac{\text{Amount of Data}}{\text{Access Time} + \text{Transfer Time}}$$

This rate is **lower than the raw transfer rate** for small data sizes due to the fixed overhead of access time.

Example:

Assume that a hard disk has:

- Access latency: 20 milliseconds = 0.02 seconds
- Transfer rate: 5 MB/s

Calculating ETR for reading 1 MB of data

1. Transfer Time: $\frac{1 \text{ MB}}{5 \text{ MB/s}} = 0.2 \text{ seconds}$
2. Total Time: $0.02 + 0.2 = 0.22 \text{ seconds}$
3. Effective Transfer Rate: $\frac{1 \text{ MB}}{0.22 \text{ s}} = 4.55 \text{ MB/s}$

Even though the disk can transfer at 5 MB/s, **the effective rate is lower** because of the 20 ms access latency.

Utilization

Utilization refers to the portion of total time actually spent transferring data.

$$\text{Utilization} = \frac{\text{Transfer Time}}{\text{Access Time} + \text{Transfer Time}}$$

$$\text{Utilization} = \frac{\frac{\text{Amount of Data}}{\text{Transfer Rate}}}{\text{Access Time} + \frac{\text{Amount of Data}}{\text{Transfer Rate}}}$$

80% utilization means,

$$0.8 = \frac{\text{Transfer Time}}{\text{Access Time} + \text{Transfer Time}}$$

$$\text{Transfer time} = 80\% \text{ of } (\text{Access Time} + \text{Transfer time})$$

Calculating Effective Transfer Rate when Seek Time and Rotational Time are given:

Formula for Effective Transfer Rate:

$$\text{Effective Transfer Rate (ETR)} = \frac{\text{Amount of Data}}{\text{Access Time} + \text{Transfer Time}(T_{\text{trans}})}$$

$$\text{Access Time} = \text{Seek Time } (T_{\text{seek}}) + \text{Rotational Latency}(T_{\text{rot}})$$

So the final formula for this case:

$$\text{ETR} = \frac{\text{Amount of Data}}{T_{\text{seek}} + T_{\text{rot}} + T_{\text{trans}}}$$

T_{seek} = Seek time (time to position the head over the correct track)

- **T_{rot} = Rotational latency (time waiting for the sector to rotate under the head)**
 - **T_{trans} = Transfer time (time to read/write the actual data)**
-

Step-by-Step Calculation Example

Given:

- HDD RPM: 7200
 - Seek Time: 8 ms (0.008 sec)
 - Number of Sectors per Track: 512
 - Sector Size: 512 bytes
 - **Rotational Latency:**
 - **Full rotation time:** $T_{\text{full}} = 60/\text{RPM} = 60/7200 = 8.33 \text{ ms}$
 - **Average rotational latency:** $T_{\text{rot}} = T_{\text{full}}/2 = 8.33/2 = 4.17 \text{ ms} (0.00417 \text{ sec})$
 - **Transfer Rate: 100 MB/s ($100 \times 10^6 \text{ bytes/sec}$)**
 - **Data Size to Transfer: 256 KB ($256 \times 1024 = 262144 \text{ bytes}$)**
-

Step 1: Calculate the Total Access Time

1. Seek Time: $T_{\text{seek}} = 8 \text{ ms} = 0.008 \text{ sec}$
 2. Rotational Latency (on average): $T_{\text{rot}} = 4.17 \text{ ms} = 0.00417 \text{ sec}$
 3. Transfer Time: $T_{\text{trans}} = \text{Data Size} / \text{Transfer Rate} = 262144 / (100 \times 10^6) = 0.00262 \text{ sec}$
-

Step 2: Compute Effective Transfer Rate

1. Total Access Time: $T_{\text{total}} = 0.008 + 0.00417 + 0.00262 = 0.01479 \text{ sec}$
 2. Effective Transfer Rate: $\text{ETR} = 262144 / 0.01479 = 17.72 \text{ MB/s}$
-

Key Observations:

- Even though the raw transfer speed of the HDD is 100 MB/s, the effective transfer rate is only ~17.72 MB/s due to seek time and rotational delays.
- The larger the file size, the closer the ETR gets to the raw transfer rate (because seek and rotational delays have less impact).
- For small random accesses, ETR drops significantly due to repeated seek time and rotational delays.